
```
//stack using array
```

```
#include <stdio.h>
```

```
#define MAX 5
```

```
int stack[MAX];
```

```
int top = -1;
```

```
void push(int value)
```

```
{
```

```
    if(top == MAX - 1)
```

```
    {
```

```
        printf("Stack Overflow\n");
```

```
    }
```

```
else
```

```
{
```

```
    top++;
```

```
    stack[top] = value;
```

```
    printf("%d inserted\n", value);
```

```
}
```

```
}
```

```
void pop()
```

```
{
```

```
    if(top == -1)
```

```
    {
```

```
        printf("Stack Underflow\n");
```

```
    }
```

```
else
```

```
{
```

```
    printf("%d deleted\n", stack[top]);
```

```
    top--;
```

```
}
```

```
void display()
{
    int i;

    if(top == -1)
    {
        printf("Stack is Empty\n");
    }
    else
    {
        printf("Stack elements are:\n");
        for(i = top; i >= 0; i--)
        {
            printf("%d ", stack[i]);
        }
        printf("\n");
    }
}

int main()
{
    push(10);
    push(20);
    push(30);

    display();

    pop();

    display();
    printf("\nPress Enter to exit...");
    getchar();

    return 0;
}
```



D:\Documents\2568\2409\20: X



10 inserted

20 inserted

30 inserted

Stack elements are:

30 20 10

30 deleted

Stack elements are:

20 10

Press Enter to exit...|

```
// queue using array
#include <stdio.h>
#define MAX 5
int queue[MAX];
int front = -1;
int rear = -1;
void enqueue(int value)
{
    if(rear == MAX - 1)
    {
        printf("Queue Overflow\n");
    }
    else
    {
        if(front == -1)
            front = 0;

        rear++;
        queue[rear] = value;
        printf("%d inserted\n", value);
    }
}
void dequeue()
{
    if(front == -1 || front > rear)
    {
        printf("Queue Underflow\n");
    }
    else
    {
        printf("%d deleted\n", queue[front]);
        front++;
    }
}
```

```
void display()
{
    int i;
    if(front == -1 || front > rear)
    {
        printf("Queue is Empty\n");
    }
    else
    {
        printf("Queue elements are:\n");
        for(i = front; i <= rear; i++)
        {
            printf("%d ", queue[i]);
        }
        printf("\n");
    }
}

int main()
{
    enqueue(10);
    enqueue(20);
    enqueue(30);

    display();

    dequeue();

    display();
    printf("\nPress Enter to exit...");
    getchar();

    return 0;
}
```



D:\Documents\h4.exe



10 inserted

20 inserted

30 inserted

Queue elements are:

10 20 30

10 deleted

Queue elements are:

20 30

Press Enter to exit...|

```
// stack using single linked list
#include <stdio.h>
#include <stdlib.h>
struct Node
{
    int data;
    struct Node *next;
};
struct Node *top = NULL;
void push(int value)
{
    struct Node *newNode;
    newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = top;
    top = newNode;
    printf("%d inserted\n", value);
}

void pop()
{
    struct Node *temp;
    if(top == NULL)
    {
        printf("Stack Underflow\n");
    }
    else
    {
        temp = top;
        printf("%d deleted\n", top->data);
        top = top->next;
        free(temp);
    }
}
```

```
void display()
{
    struct Node *temp;
    if(top == NULL)
    {
        printf("Stack is Empty\n");
    }
    else
    {
        printf("Stack elements are:\n");
        temp = top;
        while(temp != NULL)
        {
            printf("%d ", temp->data);
            temp = temp->next;
        }
        printf("\n");
    }
}

int main()
{
    push(10);
    push(20);
    push(30);
    display();
    pop();
    display();
    printf("\nPress Enter to exit...");
    getchar();

    return 0;
}
```



D:\Documents\h5.exe



10 inserted

20 inserted

30 inserted

Stack elements are:

30 20 10

30 deleted

Stack elements are:

20 10

Press Enter to exit...|

```
// queue using linked list
#include <stdio.h>
#include <stdlib.h>
struct Node
{
    int data;
    struct Node *next;
};
struct Node *front = NULL;
struct Node *rear = NULL;

void enqueue(int value)
{
    struct Node *newNode;
    newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = NULL;
    if(front == NULL)
    {
        front = rear = newNode;
    }
    else
    {
        rear->next = newNode;
        rear = newNode;
    }
    printf("%d inserted\n", value);
}

void dequeue()
{
    struct Node *temp;
    if(front == NULL)
    {
```

```

{
    printf("Queue Underflow\n");
}
else
{
    temp = front;
    printf("%d deleted\n", front->data);
    front = front->next;
    if(front == NULL)
        rear = NULL;
    free(temp);
}
}

```

```

void display()
{
    struct Node *temp;
    if(front == NULL)
    {
        printf("Queue is Empty\n");
    }
    else
    {
        printf("Queue elements are:\n");
        temp = front;
        while(temp != NULL)
        {
            printf("%d ", temp->data);
            temp = temp->next;
        }
        printf("\n");
    }
}

```

```
int main()  
{  
    enqueue(10);  
    enqueue(20);  
    enqueue(30);  
    display();  
    dequeue();  
    display();  
    printf("\nPress Enter to exit...");  
    getchar();  
    return 0;  
}
```



D:\Documents\h6.exe



10 inserted

20 inserted

30 inserted

Queue elements are:

10 20 30

10 deleted

Queue elements are:

20 30

Press Enter to exit...|

20-7-26

Stack (Array)

```
#include <stdio.h>
```

```
#define MAX 5
```

```
int stack[MAX];
```

```
int top = -1;
```

```
void push(int value) {
```

```
    if (top == MAX - 1) {
```

```
        printf("Stack overflow\n");
```

```
    }
```

```
    else {
```

```
        top++;
```

```
        stack[top] = value;
```

```
        printf("%d inserted\n", value);
```

```
    }
```

```
void pop() {
```

```
    if (top == -1) {
```

```
        printf("Stack Underflow\n");
```

```
    }
```

```
    else {
```

```
        printf("%d deleted\n", stack[top]);
```

```
        top--;
```

```
    }
```

```
void display() {
```

```
    int i;
```

```
    if (top == -1) {
```

```
        printf("Stack is empty\n");
```

```
    } else {
```

```
        printf("Stack elements are:\n");
```

```
        for (i = top; i >= 0; i--) {
```

```
            printf("%d ", stack[i]);
```

```
        }
```

```
        printf("\n");
```

```
    }
```

Output

10 inserted

20 inserted

30 inserted

Stack elements are:

30 20 10

30 (deleted)

Stack elements are:

20 10

```

int main () {
    push(10);
    push(20);
    push(30);
    display();
    pop();
    display();
    return 0;
}

```

Queue (Array)

```

#include <stdio.h>
#define MAX 5
int queue[MAX];
int front = -1;
int rear = -1;

void enqueue (int value) {
    if (rear == MAX-1) {
        printf ("Queue Overflow\n");
        return;
    }
    if (front == -1)
        front = 0;
    rear++;
    queue[rear] = value;
    printf ("%d inserted\n", value);
}

void dequeue () {
    if (front == -1 || front > rear) {
        printf ("Queue Underflow\n");
        return;
    }
    printf ("%d deleted\n", queue[front]);
    front++;
}

```

Output

10 inserted
 20 inserted
 30 inserted
 Queue elements are:
 10 20 30
 10 deleted
 Queue elements are:
 20 30

```
void display() {
```

```
    int i;
```

```
    if (front == -1 || front > rear) {
```

```
        printf("Queue is Empty\n");
```

```
    }  
    else {
```

```
        printf("Queue elements are:\n");
```

```
        for (i = front; i <= rear; i++) {
```

```
            printf("%d ", queue[i]);
```

```
        }
```

```
        printf("\n");
```

```
    }
```

```
int main() {
```

```
    enqueue(10);
```

```
    enqueue(20);
```

```
    enqueue(30);
```

```
    display();
```

```
    dequeue();
```

```
    display();
```

```
    return 0;
```

```
}
```

Stack (linked list)

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {
```

```
    int data;
```

```
    struct Node * next;
```

```
};
```

```
struct Node * top = NULL;
```

Output

10 inserted

20 inserted

30 inserted

Stack elements are:

30 → 20 → 10 → NULL

30 deleted

Stack elements are:

20 → 10 → NULL

```
void push (int value) {
```

```
    struct Node *newNode;
```

```
    newNode = (struct Node *) malloc (sizeof (struct Node));
```

```
    newNode->data = value;
```

```
    newNode->next = top;
```

```
    top = newNode;
```

```
    printf ("%d inserted \n", value);
```

```
}
```

```
void pop () {
```

```
    struct Node *temp;
```

```
    if (top == NULL) {
```

```
        printf ("Stack Underflow \n");
```

```
    }
```

```
    else {
```

```
        temp = top;
```

```
        printf ("%d deleted \n", top->data);
```

```
        top = top->next;
```

```
        free (temp);
```

```
    }
```

```
void display () {
```

```
    struct Node *temp;
```

```
    if (top == NULL) {
```

```
        printf ("Stack is Empty \n");
```

```
    }
```

```
    else {
```

```
        printf ("Stack elements are: \n");
```

```
        temp = top;
```

```
        while (temp != NULL) {
```

```
            printf ("%d", temp->data);
```

```
            temp = temp->next;
```

```
        }
```

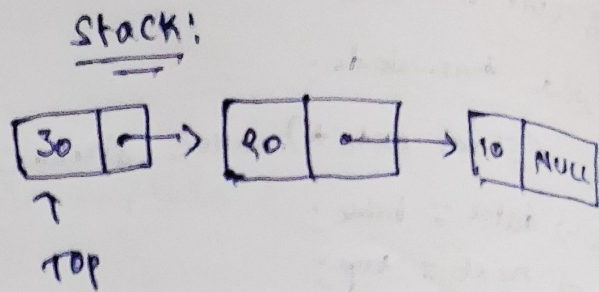
```
        printf ("\n");
```

```
}
```

```

int main() {
    push(10);
    push(20);
    push(30);
    display();
    pop();
    display();
    return 0;
}

```



Queue (linked list)

```

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node * next;
};

```

```

};

```

```

struct Node * front = NULL;
struct Node * rear = NULL;

```

```

void enqueue(int value)
{

```

```

    struct Node * newNode;
    newNode = (struct Node *) malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = NULL;

```

```

    if (front == NULL) {

```

```

        front = rear = newNode;
    }

```

```

    else {

```

```

        rear->next = newNode;

```

```

        rear = newNode;
    }

```

```

    printf("Node inserted in ", value);
}

```

```

void dequeue() {
    struct Node *temp;
    if (front == NULL) {
        printf("Queue Underflow\n");
    }
    else {
        temp = front;
        printf("%d deleted\n", front->data);
        front = front->next;
        if (front == NULL) {
            rear = NULL;
        }
        free(temp);
    }
}

```

```

void display() {
    struct Node *temp;
    if (front == NULL) {
        printf("Queue is Empty\n");
    }
    else {
        printf("Queue elements are:\n");
        temp = front;
        while (temp != NULL) {
            printf("%d ", temp->data);
            temp = temp->next;
        }
        printf("\n");
    }
}

```

```

int main() {
    enqueue(10);
    enqueue(20);
    enqueue(30);
    display();
    dequeue();
    display();
    return 0;
}

```

Output

10 inserted
20 inserted
30 inserted

Queue elements are:

10 → 20 → 30 → NULL

10 deleted

Queue elements are:

20 → 30 → NULL

